

Google Play Store Real-time Data Analysis
using Python

Abstract:

This project presents a Python based real time data analytics solution for Google Play Store. It focuses on collecting and analysing app data which includes ratings, reviews, and downloads to get relevant insights. It aims to identify trends, user preferences, and performance indicators which in turn drive app success and platform user engagement through data visualization and analysis.

Introduction:

In order to determine trends, user preferences, and elements that affect app success, this project examines data from the Google Play Store. By analysing metrics like ratings, reviews, and downloads, it aims to gain a better understanding of user engagement and application performance. Data analysis and visualization enabled by Python help developers and organizations make better decisions.

Objectives:

- To identify trends, examine user reviews, ratings, and install counts.
- Determine which app categories are the most popular and the factors influencing their expansion.
- Find trends in the preferences and actions of users.
- Display app performance metrics in an engaging and interactive manner.
- Give developers, marketers, and analysts information to help them make decisions.

Technology Stack:

- Languages: Python, HTML
- Libraries: Numpy, Pandas, Matplotlib, Seaborn, Plotly, NLTK, SKlearn
- Source: Google Play Store Dataset, User Ratings - via *Kaggle*

WORKFLOW:

Data Cleaning

Data preprocessing

Sentimental Analysis

Visualization

Creating a web dashboard

Dataset Overview:

This project utilizes two real-world Google Play Store datasets to analyse user feedback and app performance metrics, crucial for app success.

Play Store Data	User Reviews Data
This dataset includes information about over 10,800 Android applications available on the Play Store. It contains structured attributes such as: • App name • Category • Rating • Number of Reviews and Installs • App Size, Type (Free or Paid), Price • Content Rating, • Last Updated Date • Genres, Version info, and Required Android version	The second dataset complements the app data by providing over 64,000 user reviews, with a focus on translated user feedback for sentiment analysis. Each review is labelled with: • The app it belongs to • The translated review text • Sentiment classification (Positive, Neutral, or Negative) • Sentiment polarity • Sentiment subjectivity

Implementation:

1. Importing necessary libraries:

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.io as pio
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
import webbrowser
import os
```

Libraries and Their Uses:

pandas – For data manipulation

numpy – For numerical operations

plotly.express – For interactive visualizations

plotly.io – For controlling plot rendering and export

scikit-learn – For building and evaluating a machine learning model

nltk – For sentiment analysis and text processing

webbrowser – For opening URLs in a browser

os – For interacting with the operating system

2. Reading the dataset:

```
apps=pd.read_csv('Play Store Data.csv')
reviews=pd.read_csv('User Reviews.csv')
```

3. Data Cleaning:

```
apps = apps.dropna(subset=['Rating'])
for column in apps.columns :
    apps[column].fillna(apps[column].mode()[0],inplace=True)
apps.drop_duplicates(inplace=True)
apps=apps[apps['Rating']<=5]
reviews.dropna(subset=['Translated_Review'],inplace=True)
```

Removed rows with missing ratings to ensure accurate analysis of app quality:

```
apps = apps.dropna(subset=['Rating'])
```

Filled remaining missing values in all columns with the most frequent (mode) value:

```
for column in apps.columns :
    apps[column].fillna(apps[column].mode()[0],inplace=True)
```

Eliminated duplicate entries to avoid redundant or skewed results:

```
apps.drop_duplicates(inplace=True)
```

Filtered out invalid ratings greater than 5:

```
apps=apps[apps_df['Rating']<=5]
```

Dropped reviews with missing text, which are not useful for sentiment analysis:

```
reviews.dropna(subset=['Translated_Review'],inplace=True)
```

Convert the Installs columns to numeric by removing commas and +

```
apps['Installs'] = apps['Installs'].astype(str).str.replace(',', '').str.replace('+', '').astype(int)
```

Convert Price column to numeric after removing \$

```
apps['Price'] = apps['Price'].astype(str).str.replace('$', '').astype(float)
```

Before		After	
App	object	App	object
Category	object	Category	object
Rating	float64	Rating	float64
Reviews	object	Reviews	object
Size	object	Size	object
Installs	object	Installs	int32
Type	object	Type	object
Price	object	Price	float64
Content Rating	object	Content Rating	object
Genres	object	Genres	object
Last Updated	object	Last Updated	object
Current Ver	object	Current Ver	object
Android Ver	object	Android Ver	object
dtype: object		dtype: object	

Note:

Converting strings to integers or numeric formats is crucial in data preprocessing, sorting, filtering, and mathematical operations. It's also necessary for data visualizations, plotting libraries, and machine learning models, as it ensures accurate calculations and avoids incorrect or misleading results.

Merging the datasets:

```
merged_df=pd.merge(apps,reviews,on='App',how='inner')
```

- This operation merges the apps_df and reviews_df DataFrames, retaining only the rows where the App name exists in both datasets.
- An inner join is a way to combine two datasets where only the rows with matching values in the key column are kept in the result.
- pandas.merge() is similar to SQL joins, and behaves almost the same way.

Data Transformation:

The dataset's Size column contains values like "19M" for megabytes and "850k" for kilobytes, which are kept as strings. To make this column useful for numerical analysis, a custom function is used to convert these textual values into float values in megabytes.

```
def convert_size(size):
    if 'M' in size:
        return float(size.replace('M',''))
    elif 'k' in size:
        return float(size.replace('k',''))/1024
    else:
        return np.nan
apps['Size']=apps['Size'].apply(convert_size)
```

The values in the Installs column can range from a few thousand to over 10 million, making direct comparison difficult. Such a big gap makes it difficult to identify important trends in charts, and it can also impair how well machine learning algorithms learn from data. To make things easier to work with, a log transformation was performed utilizing the natural logarithm function (np.log()). This helps to reduce large numbers while maintaining relative differences. The altered values are saved in a new column named Log_Installs, making the data easier to visualize and improving the efficiency of any models built on top of it.

What does the `log()` function do?

The logarithm tells you how many times you need to multiply a number to get another number.

```
apps['Log_Installs']=np.log(apps['Installs'])  
apps['Reviews']=apps['Reviews'].astype(int)  
apps['Log_Reviews']=np.log(apps['Reviews'])
```

o/p:

```
[13] np.log(1)
```

```
np.float64(0.0)
```

```
np.log(10)
```

```
np.float64(2.302585092994046)
```

```
[15] np.log(100)
```

```
np.float64(4.605170185988092)
```

```
np.log(1000)
```

```
np.float64(6.907755278982137)
```

To make the ratings easier to understand, a new column called Rating_Group is used. It groups apps based on their ratings:

```
def rating_group(rating):  
    if rating >= 4:  
        return 'Top rated app'  
    elif rating >=3:  
        return 'Above average'  
    elif rating >=2:  
        return 'Average'  
    else:  
        return 'Below Average'  
apps['Rating_Group']=apps['Rating'].apply(rating_group)
```

A new column called Revenue is added to estimate how much money each paid app have made. It was calculated by multiplying the app's Price by the number of Installs:

```
apps['Revenue'] = apps['Price'] * apps['Installs']  
apps['Last Updated'] = pd.to_datetime(apps['Last Updated'], errors='coerce')  
apps['Year'] = apps['Last Updated'].dt.year
```

errors='coerce' means if the date is invalid or can't be read, change it to NaT (Not a Time) instead of throwing an error.

Sentimental Analysis:

In this project, sentiment analysis is used to understand how users feel about different apps based on their written reviews.

The VADER tool from the NLTK library provides a compound sentiment score between -1 and 1, with positive scores indicating positive sentiment, neutral scores indicating neutrality, and negative scores indicating negative sentiment.

```
model = SentimentIntensityAnalyzer()
def get_sentiment_score(review):
    return model.polarity_scores(str(review))['compound']
reviews['Sentiment_Score'] = reviews['Translated_Review'].apply(get_sentiment_score)
```

The function converts each review to a string and returns its compound sentiment score, which ranges from -1 (very negative) to 1 (very positive). This score was then applied to all reviews and stored in a new column called `Sentiment_Score` for further analysis.

6. File Handling and Output Directory Setup:

```
html_files_path = "./"
if not os.path.exists(html_files_path):
    os.makedirs(html_files_path)
```

To save output files safely, the code checks if the target folder exists. If not, it creates the folder. In this project, the current directory ("`./`") is used to store HTML or other generated files.

6.1 Saving Plots with Insights as HTML:

To store visualizations along with their insights, a function named `save_plot_as_html()` was created. This function saves each Plotly chart as an interactive HTML file and also collects the chart along with a written insight into a combined HTML format using a variable called `plot_containers`. This allows multiple plots and explanations to be organized in a single view.

```
plot_containers=""
def save_plot_as_html(fig, filename, insight):
    global plot_containers
    filepath = os.path.join(html_files_path, filename)
    html_content = pio.to_html(fig, full_html=False, include_plotlyjs='inline')
    # Append the plot and its insight to plot_containers
    plot_containers += f"""
        <div class="plot-container" id="{filename}" onclick="openPlot('{filename}')">
            <div class="plot">{html_content}</div>
            <div class="insights">{insight}</div>
        </div>
    """
    fig.write_html(filepath, full_html=False, include_plotlyjs='inline')
plot_width=400
plot_height=300
plot_bg_color='black'
text_color='white'
title_font={'size':16}
axis_font={'size':12}
```

Visualization:

Plotly's Python graphing library makes interactive, publication-quality graphs.

Static visualization libraries like Matplotlib and Seaborn generate fixed charts used mainly in reports and publications, offering limited user interaction. In contrast, Plotly creates interactive visualizations that allow users to hover, zoom, and explore data dynamically, making it ideal for web dashboards and real-time data analysis. While static libraries are great for print-ready visuals, Plotly is better suited for engaging, web-based visual storytelling.

#Figure 1:

```
custom_colors = [
    '#a7b195', '#8e9761', '#8b914a', '#4a693f', '#3e5a3a', '#4c6347', '#75805a', '#888e63', '#898b5e',]
category_counts=apps['Category'].value_counts().nlargest(10)
fig1 = px.bar(
    x=category_counts.index,
    y=category_counts.values,
    labels={'x': 'Category', 'y': 'Count'},
    title='Top Categories on Play Store',
    color=category_counts.index,
    color_discrete_sequence=custom_colors,
    width=400,
    height=300
)
fig1.update_layout(
    plot_bgcolor='rgba(255, 255, 255, 0.2)',
    paper_bgcolor='black',
    font_color='white',
    title_font={'size': 16},
    xaxis=dict(title_font={'size': 12}),
    yaxis=dict(title_font={'size': 12}),
    margin=dict(l=10, r=10, t=30, b=10)
)
save_plot_as_html(
    fig=fig1,
    filename="Category Graph 1.html",
    insight="Tools, entertainment, and productivity apps are the most popular types of apps on the Play Store."
)
```

#Figure 2:

```
type_color_palette = ['#8b914a', '#3e5a3a'] # Conifer & Fern

type_counts = apps['Type'].value_counts()

fig2 = px.pie(
    values=type_counts.values,
    names=type_counts.index,
    title='App Type Distribution',
    color_discrete_sequence=type_color_palette,
    width=400,
    height=300
)

fig2.update_layout(
    plot_bgcolor='rgba(255, 255, 255, 0.2)', # soft glassy white
    paper_bgcolor='black',
    font_color='white',
    title_font={'size': 16},
    margin=dict(l=10, r=10, t=30, b=10)
)

save_plot_as_html(
    fig2,
    "Type Graph 2.html",
    "Most apps on the Playstore are free, indicating a strategy to attract users first and
monetize through ads or in-app purchases."
)
```

Figure 3:

```
rating_color = ['#8e9761'] # Citron

fig3 = px.histogram(
    apps,
    x='Rating',
    nbins=20,
    title='Rating Distribution',
    color_discrete_sequence=rating_color,
    width=400,
    height=300
)
fig3.update_layout(
    plot_bgcolor='rgba(255, 255, 255, 0.2)',
    paper_bgcolor='black',
    font_color='white',
    title_font={'size': 16},
    xaxis=dict(title_font={'size': 12}),
    yaxis=dict(title_font={'size': 12}),
    margin=dict(l=10, r=10, t=30, b=10)
)
save_plot_as_html(
    fig3,
    "Rating Graph 3.html",
    "Ratings are skewed towards higher values, suggesting that most apps are rated favorably by users."
)
```

#Figure 4:

```
sentiment_green_palette = ['#a7b195', '#8e9761', '#4c6347', '#75805a', '#8b914a']
sentiment_counts=reviews['Sentiment_Score'].value_counts()
fig4 = px.bar(
    x=sentiment_counts.index,
    y=sentiment_counts.values,
    labels={'x': 'Sentiment Score', 'y': 'Count'},
    title='Sentiment Distribution',
    color=sentiment_counts.index,
    color_discrete_sequence=sentiment_green_palette,
    width=400,
    height=300
)
fig4.update_layout(
    plot_bgcolor='rgba(255, 255, 255, 0.2)',
    paper_bgcolor='black',
    font_color='white',
    title_font={'size': 16},
    xaxis=dict(title_font={'size': 12}),
    yaxis=dict(title_font={'size': 12}),
    margin=dict(l=10, r=10, t=30, b=10)
)

save_plot_as_html(
    fig4,
    "Sentiment Graph 4.html",
    "Sentiments in reviews show a mix of positive and negative feedback, with a slight
    lean towards positive sentiments"
)
```

Creating a Web Dashboard:

```
plot_containers_split=plot_containers.split('</div>')
```

It splits the full HTML string (plot_containers) at every </div> tag.

```
if len(plot_containers_split) > 1:  
    final_plot=plot_containers_split[-2]+'</div>'  
else:  
    final_plot=plot_containers
```

This code splits the plot_containers string wherever appears to determine whether there are several sections inside it. If so, it takes the second-to-last portion and adds the back, extracting and returning only the final full plot block. It just preserves the original content if there is just one or none. This is mostly used to isolate or preview the most recent plot addition.

Dashboard.html:

```
dashboard_html = ""
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Google Play Store Review Analytics</title>
  <style>
    body {{
      font-family:Times new roman;
      background-color: #ffffff;
      color: #000000;
      margin: 0;
      padding: 0;
    }}
    .header {{
      display: flex;
      align-items: center;
      justify-content: center;
      padding: 20px;
      background-color: #ffffff;
      flex-wrap: wrap;
    }}
    .header img {{
      margin: 0 10px;
      height: 50px;
    }}
    .header h1 {{
      font-size: 24px;
      margin: 0 10px;
      text-align: center;
```

```
        color: #000000;
    }}
    .container {{
        display: flex;
        flex-wrap: wrap;
        justify-content: center;
        align-items: flex-start;
        padding: 20px;
        gap: 20px;
    }}
    .plot-container {{
        border: 2px solid #555
        margin: 10px;
        padding: 10px;
        width: {plot_width}px;
        height: {plot_height}px;
        overflow: hidden;
        position: relative;
        cursor: pointer;
    }}
    .insights {{
        display: none;
        position: absolute;
        right: 10px;
        top: 10px;
        background-color: rgba(0,0,0,0.7);
        padding: 5px;
        border-radius: 5px;
        color: #fff;
    }}
    .plot-container: hover .insights {{
        display: block;
    }}
</style>
```

```
<script>
  function openPlot(filename) {{
    window.open(filename, '_blank');
  }}
</script>
</head>
<body>
  <div class="header">
    <h1>Google Play Store Reviews Analytics</h1>
  </div>
  <div class="container">
    {plots}
  </div>
</body>
</html>
"""
```

This HTML page shows visual insights from Google Play Store data. It includes charts about app ratings, categories, installs, user reviews, and sentiments. The dashboard helps understand how apps perform and what users feel about them in a clean and interactive way.

```
final_html=dashboard_html.format(plots=plot_containers,plot_width=plot_width,plot_height=plot_height)
```

This line takes the HTML template and fills in the plot area and size. It replaces {plots} with all the saved charts, and also sets the width and height using {plot_width} and {plot_height}. The final result is stored in final_html, which is the complete dashboard ready to be saved or opened in a browser.

```
dashboard_path=os.path.join(html_files_path,"index.html")
```

This line creates the full path where the HTML file will be saved. It joins the folder path (html_files_path) with the file name ("index.html") and stores the result in dashboard_path. This makes sure the file is saved in the correct location with the right name.

```
with open(dashboard_path, "w", encoding="utf-8") as f:  
    f.write(final_html)
```

This line opens the file at dashboard_path in write mode with UTF-8 encoding. Then it writes the complete HTML content stored in final_html to that file. After this, your dashboard is saved and ready to be opened in a browser.

```
webbrowser.open('file://'+os.path.realpath(dashboard_path))
```

This line opens the saved HTML dashboard in your default web browser. It converts the file path to a full URL using os.path.realpath and uses web browser.open to launch it automatically.

Conclusion:

In conclusion, this project provided valuable insights into the Google Play Store through real-time data analysis and visualization. By exploring app categories, user ratings, installs, and sentiment analysis, the dashboard helps understand key trends and user behavior. It also demonstrates how Python and Plotly can be used effectively to turn raw data into meaningful, interactive visuals.